

湖南工商大学

HUNAN UNIVERSITY OF TECHNOLOGY AND BUSINESS

《人工智能专业实习报告》



姓 名:	陈佳婷
学 号:	2227030079
专 业:	人工智能(数字媒体与 AI 院士卓越班)
班 级:	智媒体 2202 班
指导教师:	张震
职 称:	讲师

前沿交叉学院

2023 年 6 月

基于深度学习的肝脏肿瘤分割智能辅助诊断系统

1. 引言

1.1 项目背景与研究动机

人工智能技术自 2012 年开始迎来快速发展的繁荣景象，当前正处于行业广泛研发及快速落地商业化阶段^[1]。随着医疗技术和计算能力的不断提升，智能医疗逐渐成为现代医疗发展的重要方向之一。尤其是在医学影像分析领域，人工智能（AI）技术的应用为临床诊断带来了显著的效率提升和精度改进。肝脏肿瘤作为一种常见且致命的疾病，早期的准确诊断对于患者的治疗和预后具有关键作用。然而，传统的肝脏肿瘤诊断依赖于人工分析，耗时且易受主观因素影响，存在一定的误诊和漏诊风险。为此，基于深度学习的智能诊断系统逐渐成为研究热点，旨在通过自动化的图像分析技术，辅助医生进行更准确和高效的诊断。

1.2 智能医疗和肝脏肿瘤诊断的现状

当前，智能医疗的发展已覆盖多个领域，在人工智能的加持下实现从呈现高清影像图片到提供高准确度的辅助诊断，并正在向自动诊断前进^[1]。深度学习中的卷积神经网络、堆叠自动编码器、生成对抗网络等图像特征识别算法不断改进医学影像技术水平，不仅可以自动实现图像自动分割，还可以实现疾病的自动诊断。从病情预测到个性化治疗方案，AI 技术在医疗中的应用日益广泛。特别是在肝脏肿瘤诊断中，医学影像（如 CT 和 MRI）的分析是诊断的关键环节。该技术被应用到医学领域，可协助医生准确、快速地筛选、发现、标记细微病灶，辅助临床对疾病作出诊断，降低漏诊率、误诊率^[2]。

1.3 智能诊断系统与深度学习在肝脏肿瘤分割中的应用场景以及挑战

在肝脏肿瘤的自动分割任务中，深度学习模型，特别是卷积神经网络（CNN），表现出了强大的特征提取和分类能力。通过对大量标注数据的训练，这些模型能够准确地分割出肿瘤区域，辅助医生进行病灶定位和诊断。然而，肝脏肿瘤分割仍然面临一些挑战，如肿瘤形状的不规则性、边界模糊、数据集不均衡等，这些问题都会影响模型的表现。因此，如何设计一个鲁棒性强、适应性好的深度学习模型，是当前研究的重要方向。

1.4 本研究的目标和内容

本研究旨在设计并实现一个基于深度学习的肝脏肿瘤自动分割智能诊断系统。该系统将利用 3D-IRCADb-01 数据集，通过卷积神经网络（如 U-Net）的训练，自动识别并分割出肝脏肿瘤区域，为临床诊断提供辅助支持。本研究的主要内容包括数据集的处理与

预处理、深度学习模型的设计与优化、模型的训练与评估、以及最终系统的部署与应用。通过本研究，我们希望为肝脏肿瘤的早期诊断提供一个高效、可靠的工具，提高临床诊断的准确性和效率。

2. 项目需求分析

2.1 用户需求

肝脏肿瘤的早期检测对于患者的治疗效果至关重要，因此，临床医生迫切需要一种高效、准确的工具来辅助肿瘤的检测与分割。传统的手工标注和分析方法不仅耗时长、效率低，还容易受到人为因素的影响，导致结果的准确性无法保证。因此，开发一个基于深度学习的自动分割系统，可以帮助医生更快地识别肿瘤区域，提高诊断的效率和精度。用户需求主要集中在以下几个方面：

- (1) **准确性：**系统应能够准确识别肝脏内的肿瘤，并精确分割肿瘤的边界。
- (2) **效率：**系统应在短时间内处理大量医学影像，提供实时或近实时的诊断结果。
- (3) **易用性：**系统界面应简洁明了，易于医生操作和理解，支持图像的上传和结果的可视化展示。
- (4) **兼容性：**系统应兼容不同的医学影像格式，如 DICOM、JPEG 等，适用于多种临床环境。

2.2 功能需求

为了满足用户需求，肝脏肿瘤智能辅助分割系统需要具备以下核心功能：

- (1) **医学影像上传与预处理：**允许用户上传医学影像文件（如 CT、MRI 扫描），并将这些图像存储在安全的数据库中，上传的图像需要经过预处理，包括归一化、尺寸调整和噪声去除，以确保数据质量并优化模型的输入效果。
- (2) **肿瘤自动分割：**基于训练好的深度学习模型，系统自动对上传的医学影像进行肝脏肿瘤的分割，输出肿瘤区域的分割结果。
- (3) **结果可视化：**系统应提供分割结果的可视化展示功能，医生可以直观地看到原始图像与分割图像的对比。
- (4) **诊断报告生成：**系统自动生成包含分割结果的诊断报告，供医生参考或存档。
- (5) **数据管理与存储：**提供安全的数据存储功能，保存上传的医学影像、分割结果以及生成的报告。

2.3 技术需求

为了实现上述功能，系统需满足以下技术需求：

- (1) **算法：**核心分割任务将使用卷积神经网络（如 U-Net）进行训练和推理。此外，可能需要其他辅助算法，如图像增强技术、后处理算法（如形态学处理）等。

(2) 计算资源: 深度学习模型的训练需要大量的计算资源, 建议使用高性能 GPU 进行模型训练, 以加速训练过程。同时, 系统的推理过程应在 CPU 或 GPU 上高效运行, 确保实时性。

(3) 数据类型: 系统应处理 3D 医学影像数据 (如 CT、MRI), 并兼容常见的医学图像格式 (如 DICOM)。数据预处理包括图像归一化、尺寸调整、数据增强等步骤。

(4) 平台兼容性: 系统应设计为跨平台应用, 能够在多种操作系统上运行, 如 Windows、Linux 等。为了实现模型的高效部署, 可以考虑使用 Docker 容器化技术。

3. 数据集与预处理

3.1 数据集介绍

3D-IRCADb-01^[3]数据集是一个专门用于研究肝脏和肝脏病变 (如肿瘤) 的高质量医学影像数据集。该数据集由法国 IRCAD 研究中心提供, 包含从多名患者收集的 CT 扫描图像, 涵盖了不同类型和大小的肝脏肿瘤。这些数据广泛应用于医学影像分析领域的研究, 如肿瘤检测、分割和分类。

(1) 数据来源: 该数据集来自真实的临床病例, 包含高分辨率的腹部 CT 扫描图像。

下载网址: <https://www.ircad.fr/research/data-sets/liver-segmentation-3d-ircadb-01/>

(2) 数据类型与结构: 该数据库包含了 10 名女性和 10 名男性的肝脏 CT 扫描数据, 其中 75% 的病例具有肝肿瘤。每个数据集都保存在一个名为 `3D-IRCADb-01-number` 的文件夹中, 其中 `number` 代表患者编号 (01 到 20 之间)。每个患者的文件夹下包括以下四个子文件夹: PATIENT_DICOM (存放患者的 DICOM 格式的匿名图像)、LABELLED_DICOM (包含分割后的感兴趣区域 (如肝脏、肿瘤等) 的标签图像)、MASKS_DICOM (包含各感兴趣区域的掩码图像)、MESHERS_VTK (包含感兴趣区域表面的网格文件 (VTK 格式))。

(3) 数据多样性: 数据集中涵盖了不同类型、大小和形状的肿瘤, 可以用于评估肝脏分割软件在处理因邻近器官接触、肝脏形状或密度异常等情况时遇到的主要困难, 提供了丰富的样本用于训练和验证深度学习模型。

3.2 数据预处理

3.2.1 数据读取

我使用了 pydicom 库批量读取 3D-IRCADb-01 数据集中的 CT 扫描图像, 并根据 InstanceNumber 对图像切片进行排序, 确保图像切片的顺序与扫描顺序一致, 同时将读取的图像切片转换为 Numpy 数组, 以便进行后续的处理。

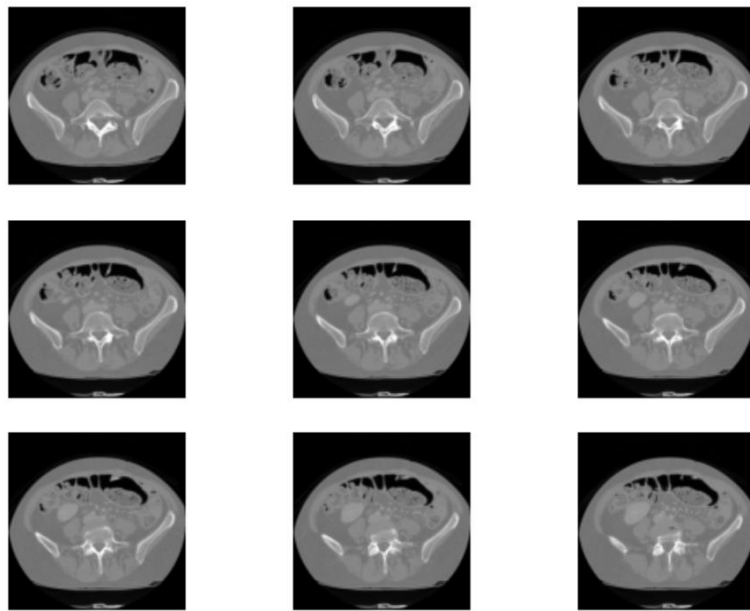


图 1 前 9 张肿瘤 CT 图

3.2.2 图像增强——Windowing 方法、直方图均衡法

首先使用 Windowing 方法，该方法用于调整 CT 图像的灰度范围，以突出显示特定的组织结构。CT 图像的像素值代表不同组织的放射密度，通过设置窗宽和窗中心，可以调整图像的对比度，使得感兴趣的区域（如肝脏和肿瘤）在图像中更加明显。在本项目中，我定义了一个 windowing 函数，使用窗宽 250 和窗中心 0，对原始的 CT 图像进行了对比度增强。这一操作有助于提高模型对肿瘤区域的识别能力，特别是在肿瘤边界模糊或对比度较低的情况下。

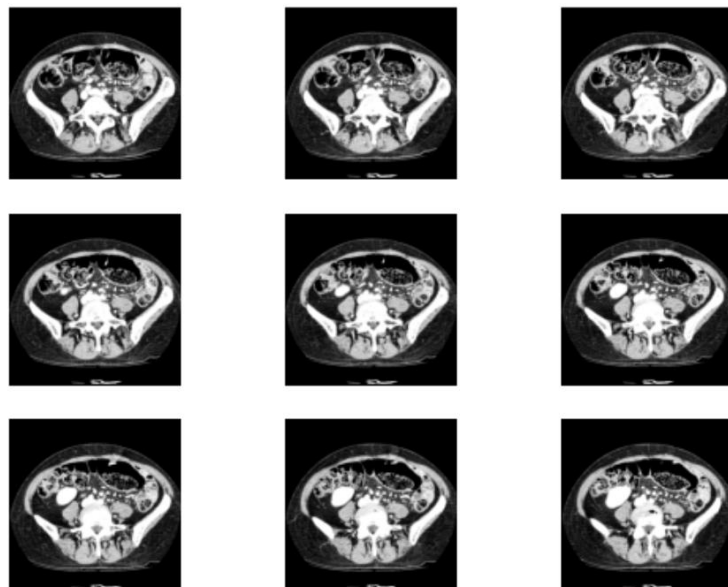


图 2 前 9 张使用 Windowing 增强后的肿瘤 CT 图

接下来，我使用了 CLAHE（对比度受限自适应直方图均衡化）算法进一步增强图像的局部对比度。传统的直方图均衡化方法虽然可以整体提升图像对比度，但在处理不均匀照明或存在局部亮度变化的医学图像时，容易导致图像质量的下降。CLAHE 通过在局部区域内进行直方图均衡化，有效避免了这种问题，特别适用于 CT 图像的处理。在实际操作中，我将 CLAHE 算法应用于 windowing 增强后的 CT 图像，显著提升了肿瘤区域的可见性，确保模型能够更准确地分割肿瘤区域。

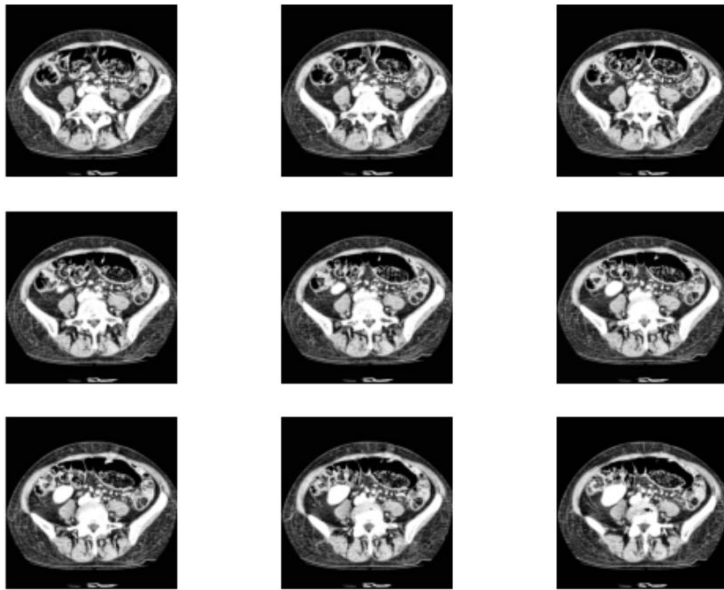


图 3 前 9 张使用直方图均衡法增强后的肿瘤 CT 图

3.2.3 肿瘤掩膜处理

在医学影像分割任务中，肿瘤掩膜处理是确保模型能够正确识别和分割肿瘤区域的重要步骤。掩膜图像通常包含了目标区域的精确标注，是模型训练过程中必不可少的监督信号。本研究的肿瘤掩膜处理过程主要包括掩膜图像的读取与处理、以及与 CT 图像的匹配。

首先，为了获取与 CT 图像相对应的肿瘤掩膜图像，我通过 pydicom 库批量读取了 3D-IRCADb-01 数据集中预标注的肿瘤掩膜数据。每个肿瘤掩膜图像与 CT 图像一一对应，并且同样根据 InstanceNumber 进行排序，确保图像切片的顺序与原始 CT 图像一致。通过这种方式，我能够提取出每个切片中的肿瘤区域，为后续的模式训练提供精确的标签。在读取并处理了肿瘤掩膜图像后，我进行了肿瘤掩膜与增强后的 CT 图像的匹配处理。为了确保模型能够从对齐的 CT 图像和掩膜图像中学习到有效的特征，我动态生成了一个布尔索引数组，用于选择与 CT 图像长度匹配的掩膜图像切片。这一步骤非常关键，因为 CT 图像和掩膜图像必须严格对齐，才能确保训练过程中模型获取的输入和标签之间的对应关系准确无误。通过这种匹配处理，我确保了数据集中的每一对图像和标签都是正确对应的，为模型的高效训练奠定了基础。

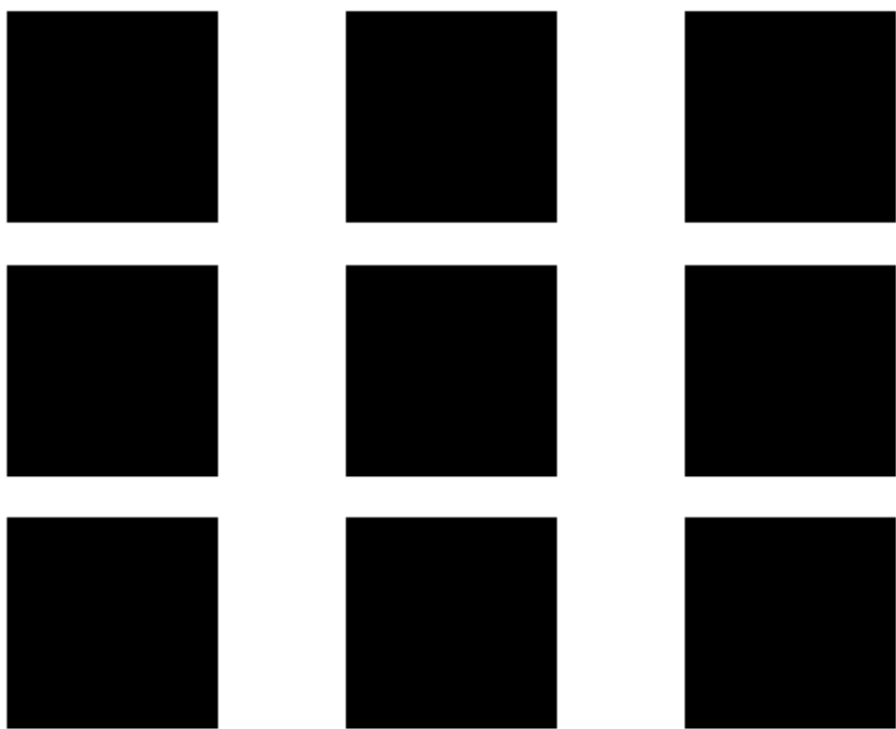


图 4 前 9 张肿瘤掩膜图

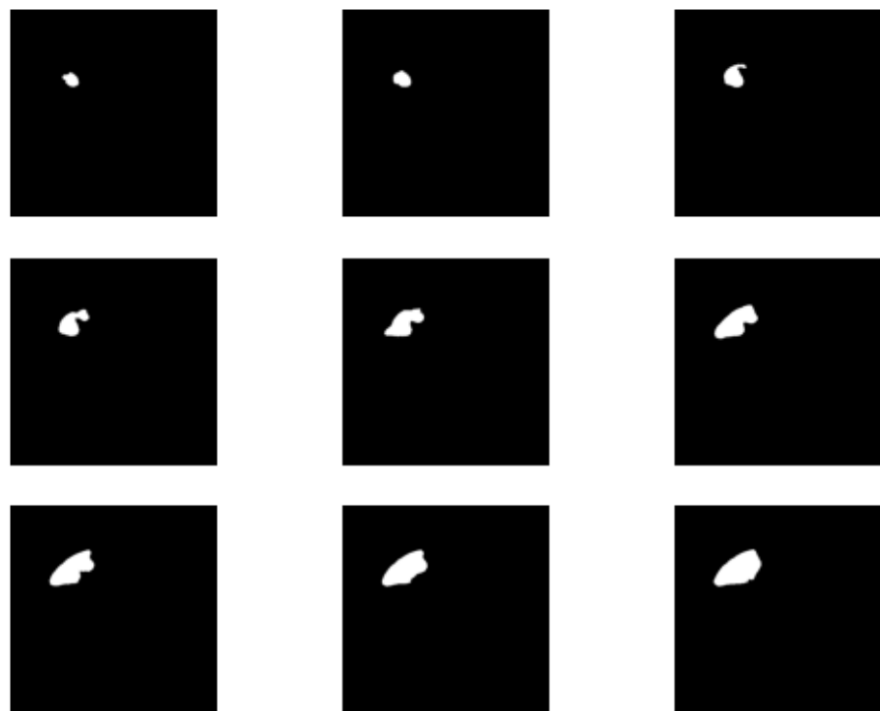


图 5 后 9 张肿瘤掩膜图

由图我们可知该病人前期是无肿瘤的，后期逐渐长出肿瘤。

3.2.4 数据保存

我根据不同的患者编号和肿瘤编号，定义并创建保存路径 `patient` 和 `tumor`，确保处理后的数据得到妥善存储。同时使用 `plt.imsave` 将处理后的 CT 图像和肿瘤掩膜图像保存为 .jpg 格式，存储在指定路径中，便于后续模型的训练和测试使用。

3.2.5 数据增强与归一化

在本研究中，我使用了 `ImageDataGenerator` 图片生成器进行图像数据增强。`ImageDataGenerator` 是 TensorFlow 中的一个强大工具，它能够对图像数据进行实时的数据增强处理，如旋转、平移、缩放等。这些变换可以有效地模拟不同的成像条件和患者姿态，从而增强模型的鲁棒性。具体来说，我使用该工具使得 CT 图像和相应的肿瘤掩膜图像在每次训练时都能够被随机变换。随后，我使用 `flow_from_directory` 方法从指定目录中加载图像数据，批量生成增强后的图像，并将其存储在目标路径中。这一过程不仅扩展了训练数据集的多样性，还提高了模型对图像变形和噪声的适应能力。

为了确保增强后的图像能够适应深度学习模型的输入要求，我对图像进行了归一化处理。在图像增强后的处理步骤中，我编写了一个 `adjust_data` 函数，将 CT 图像和肿瘤掩膜图像的像素值归一化到 $[0, 1]$ 区间。对于掩膜图像，我进一步将像素值二值化，设置阈值为 0.5，将大于阈值的像素设为 1，低于阈值的设为 0。这一步骤可以确保肿瘤区域在掩膜图像中清晰可见，并且避免了灰度值对模型学习过程的干扰。归一化后的图像能够以统一的格式输入到模型中，从而提高模型训练的稳定性和收敛速度。

4. 深度学习模型技术分析

在肝脏肿瘤分割任务中，选择合适的深度学习模型并优化其训练过程是确保分割效果的关键。本研究中，我们采用了 U-Net^[4] 深度学习模型，这些模型在医学图像分割领域表现出了良好的性能。以下是模型架构的选择、训练细节及优化策略的详细分析。

4.1 模型架构选择与设计

U-Net 是一种常用于医学图像分割的全卷积神经网络，因其 U 形结构而得名。U-Net 模型由编码器（下采样路径）和解码器（上采样路径）组成，其中编码器用于提取图像特征，解码器用于恢复图像的空间分辨率并生成分割结果。通过跳跃连接，编码器的特征图可以直接传递给解码器，保留了图像中的细节信息。由于肝脏肿瘤的形状复杂且边界模糊，U-Net 的这种架构非常适合提取和保留肿瘤区域的精细特征。

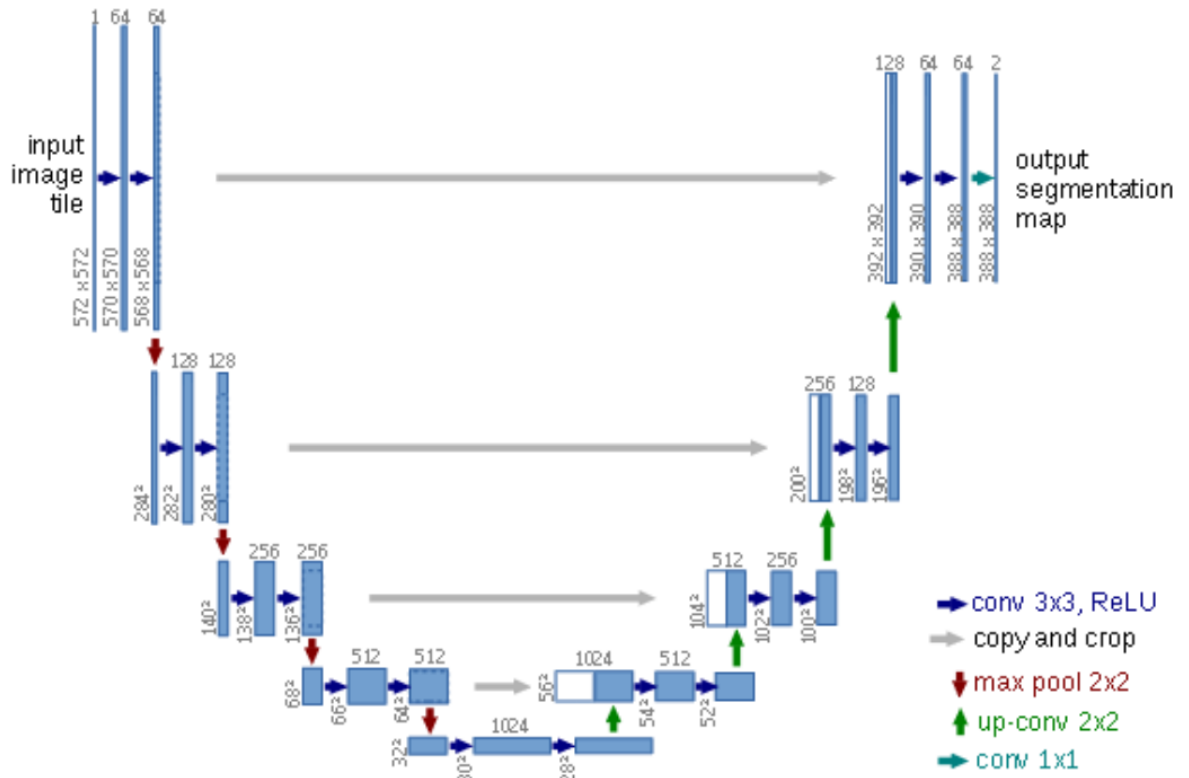


图 6 U-Net 网络结构图

在本研究中，我们设计的 U-Net 模型具体包括以下几部分：

(1) **输入层：**模型接收尺寸为(512, 512, 1)的输入图像，这是经过预处理后的 CT 图像。输入图像的单通道表示灰度图像。

(2) **编码器部分：**通过连续的卷积层（Conv2D）和池化层（MaxPool2D），逐步将图像尺寸减半，同时增加特征图的深度。卷积层使用 ReLU 激活函数，卷积核大小为 3x3，填充方式为 same，以确保图像边界的像素得到处理。池化层采用 2x2 的核进行最大池化。第 1 至 4 层：每层包括两次卷积操作和一次池化操作，逐步提取更高层次的特征。特征图深度依次增加为 64、128、256 和 512，编码器最终提取到 1024 个深度的特征图。

(3) **瓶颈层：**这是编码器和解码器之间的过渡层，采用更深的卷积操作（1024 个卷积核），用以提取全局特征。

(4) **解码器部分：**通过上采样层（UpSampling2D）逐步恢复图像的空间分辨率，并将编码器的特征图通过跳跃连接传递到对应的解码层。每次上采样后，特征图的深度依次减小至 512、256、128 和 64。解码器通过卷积操作将合并后的特征图进行处理，恢复到与输入图像相同的空间分辨率。

(5) **输出层：**最后，通过一个 1x1 卷积层将特征图的通道数降为 1，并通过 sigmoid 激活函数将输出值归一化到[0, 1]之间，生成最终的肿瘤分割掩膜。

4.2 模型训练

在 U-Net 模型的训练过程中，我们使用了 `model.fit` 函数来启动和控制训练过程。`model.fit` 函数是 Keras 中训练深度学习模型的核心方法，它能够迭代地优化模型参数，直到模型收敛到最佳状态。

(1) **训练参数**：使用了一个数据生成器 `gene` 来动态生成训练数据。这个生成器在训练过程中不断地提供 CT 图像和对应的肿瘤掩膜，以批量形式输入模型。将每个周期的步数设置为 50，训练周期为 50，同时使用了两个回调函数(`ModelCheckpoint`、`ShowMask`)。

(2) **损失函数**：使用的是二元交叉熵 `binary_crossentropy`，它通过计算预测概率与真实标签的对数损失，来衡量模型的预测准确性。

(3) **优化器**：使用的是自适应矩估计 `Adam` 能够自动调整学习率，适应不同的梯度。加快训练过程并提高收敛性。

(4) **评估指标**：使用的是准确率 `accuracy`，评估模型在训练和验证集上的表现。

4.3 模型评估与结果可视化

在本研究中，我们训练了基于 U-Net 架构的肝脏肿瘤分割模型，并对其进行了全面的评估与可视化展示。模型训练过程中，随着训练轮数 (`epochs`) 的增加，模型的准确率逐步提高，损失值逐步降低。平均准确率高达 98%，在第 11 个 `epoch` 时，模型取得了最佳性能，训练集的准确率达到 99.69%，损失值为 0.0299，验证集的准确率为 96.25%，损失值为 0.1371。这些指标显示模型在学习过程中已经很好地拟合了训练数据，并且在验证数据上保持了较高的泛化能力。

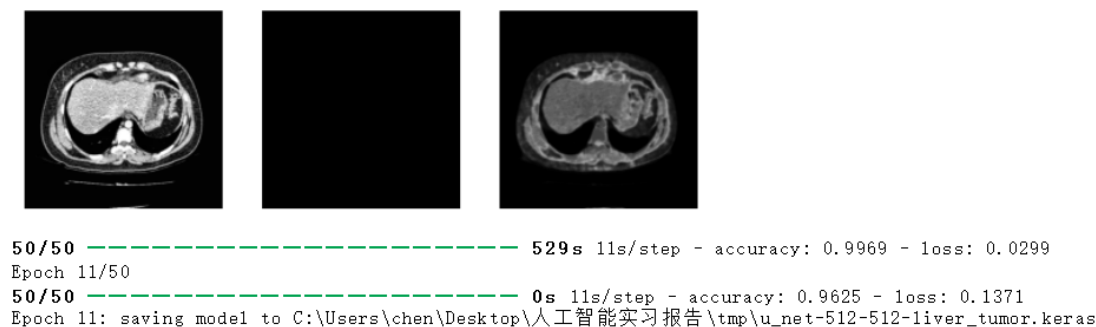


图 6 U-Net 模型训练最优结果图

在模型评估过程中，我们还进行了模型的结果可视化分析。通过每个 `epoch` 结束后的自定义回调函数 `ShowMask`，我们可以对模型的预测掩码与实际掩码进行对比。上图中，左侧展示的是原始 CT 图像，中间为实际的肝脏肿瘤掩码，而右侧展示的是模型预测的掩码。可以看出，模型的预测与实际掩码高度吻合，说明模型在肿瘤区域的定位和分割方面表现良好。

4.4 模型的改进与优化措施

在模型训练的基础上，我们还对模型进行了多项改进与优化措施。首先，我们引入了卷积神经网络中的 Batch Normalization 层，用于加速训练过程并提高模型的稳定性。其次，我们在解码器部分引入了 Dropout 层，以减少模型的过拟合现象，特别是在处理复杂的肿瘤分割任务时。此外，我们还尝试了不同的学习率调度策略，如余弦退火学习率和周期性学习率，这些策略能够进一步优化模型的收敛速度和分割性能。

5. 模型部署

在完成 U-Net 模型的训练和评估后，模型的部署成为将其应用于实际临床环境的重要一步。为了能够在实际应用中有效地使用该模型，需要将其部署到生产环境中。部署过程涉及到技术栈的选择、从开发环境到生产环境的迁移，以及在云端环境中的运行与测试等多个环节。以下是本研究的模型部署方案。

为了确保模型能够在生产环境中高效稳定地运行，本次模型部署选用了阿里云作为主要的云计算平台。阿里云提供了丰富的计算资源、灵活的伸缩性以及强大的数据处理能力，能够满足本次项目对大规模数据处理和实时响应的需求。为了进一步增强模型的可维护性和可扩展性，Docker 容器技术和 Kubernetes 容器编排系统也被纳入了技术栈中。Docker 能够将模型及其依赖打包成轻量级的容器，从而确保模型在不同环境中的一致性。而 Kubernetes 则能够自动化管理容器的部署、伸缩和运维，提升系统的容错能力。

5.1 部署流程

5.1.1 开发环境准备

- (1) 在本地环境中完成模型的开发与初步测试，确保模型性能达标。
- (2) 使用 Docker 将模型及其依赖打包成容器镜像，镜像包含 TensorFlow、OpenCV 等必要的库和工具。

5.1.2 容器化模型

- (1) 在阿里云的容器服务（ACK）中创建 Docker 镜像仓库，并将本地构建的模型镜像推送至该仓库。
- (2) 使用 Kubernetes 编写部署脚本（YAML 文件），定义 Pod、Service 等资源。

5.1.3 生产环境迁移

- (1) 在阿里云的 Kubernetes 集群中部署模型，利用 Kubernetes 的自动伸缩功能确保模型能够应对流量波动。

- (2) 配置负载均衡和自动重启策略，确保服务的高可用性。

5.1.4 运行与测试

- (1) 在阿里云上启动容器并运行模型，对接前端或 API 服务进行预测。
- (2) 使用阿里云的监控服务（如 CloudMonitor）实时监控模型的运行状态和资源使用情况。

5.1.5 效果评估与优化

- (1) 基于监控数据和实际效果，持续评估模型的预测准确率和响应速度。
- (2) 通过 Kubernetes 进行资源调配，调整模型的副本数量，以提升服务质量。
- (3) 根据需求对 Docker 镜像进行优化，减少镜像体积，提高加载速度。

5.2 实际应用中的挑战与未来改进

在实际应用中，尽管模型系统能够在本地环境中运行，但仍然面临一些挑战，这些挑战为未来的改进提供了方向。

- (1) 数据多样性不足：由于训练数据的来源较为单一，模型在面对不同分辨率、噪声水平和扫描条件的 CT 图像时，可能会表现出适应性不足的现象。未来可以通过收集更多元的训练数据，或者使用数据增强技术，进一步提升模型的泛化能力。
- (2) 实时性要求：在高并发环境下，尽管使用了负载均衡和容器化技术，模型的响应速度仍然存在瓶颈。未来可以考虑优化模型结构，减少推理时间，或者使用更强大的计算资源（如 GPU 集群）来提升实时处理能力。
- (3) 模型解释性不足：目前模型的预测结果仅表现为二值化的肿瘤掩膜，缺乏对预测结果的解释。未来可以结合可解释性 AI 技术，生成更详细的分割报告，帮助医生理解模型的判断依据。

6. 应用实现

6.1 前端功能调研

前端是用户与模型交互的界面，主要功能包括：

- (1) 用户登录与身份验证：提供用户注册、登录和身份验证功能，确保只有授权用户可以访问模型的肝脏肿瘤分割功能。
- (2) CT 图像上传与预处理：用户可以通过前端界面上传 CT 图像文件。前端需要对图像进行初步检查，如文件格式和大小的验证，并在必要时对图像进行预处理（如裁剪、缩放）以符合模型的输入要求。

(3) 实时分割结果展示：在用户上传 CT 图像后，前端需要接收来自后端的分割结果，并将其直观地显示在用户界面上。可使用图像叠加技术将分割掩膜与原始 CT 图像叠加在一起，并提供多种视图模式（如三维视图或多切片视图）。

(4) 报告生成与下载：前端还应提供分割结果的分析报告功能，用户可以生成详细的分割报告，并以 PDF 等格式下载。报告内容包括分割结果的统计信息，如肿瘤体积、位置和数量等。

(5) 用户反馈与支持：提供用户反馈通道，让用户可以对模型结果或系统使用情况提出反馈。此外，还需要有在线帮助或支持功能，为用户解答常见问题或提供使用指南。

6.2 后端功能调研

后端是模型的核心计算与数据处理部分，主要功能包括：

(1) 用户管理与权限控制：负责用户信息的存储与管理，并实现基于角色的访问控制，确保不同用户具有适当的权限来访问和操作系统。

(2) CT 图像存储与管理：需要一个高效的图像存储系统来管理用户上传的 CT 图像，以及分割结果的存储与管理。可以使用云存储解决方案，如阿里云的 OSS(对象存储服务)，以确保数据的持久性和安全性。

(3) 肿瘤分割模型的调用：后端负责将前端上传的 CT 图像传递给深度学习模型进行处理，并将模型的分割结果返回给前端。为保证高效处理，可以使用 API 服务来与前端进行通信，如 RESTful API 或 GraphQL。

(4) 任务队列与并发处理：在高并发环境下，为了保证系统响应速度和稳定性，后端需要实现任务队列和并发处理机制。可以采用消息队列系统（如 RabbitMQ）和异步任务处理框架（如 Celery）来管理和分配任务。

(5) 日志记录与异常处理：后端需要实现详细的日志记录与异常处理机制，以便在系统出现问题时能够及时定位和解决。可以使用阿里云的日志服务来集中管理和分析日志信息。

(6) 安全与数据隐私：在处理医疗数据时，确保数据的安全性与隐私保护尤为重要。后端需要实现数据加密、访问审计等功能，并遵守相关的数据隐私法律法规（如 GDPR）。

6.3 技术选型

在实现上述功能时，需要针对前后端的需求选择合适的技术栈：

6.3.1 前端技术选型

(1) 框架：推荐使用 React 或 Vue.js 作为前端框架，这些框架有丰富的组件和工具支持，可以快速构建响应式和动态的用户界面。

(2) 图像处理：可以使用 OpenCV.js 来处理 CT 图像的预处理与结果展示。为了增强用户体验，还可以引入 D3.js 或 Three.js 进行三维图像渲染。

(3) 样式框架：可以使用 Bootstrap 或 Tailwind CSS 等前端样式框架来快速设计和美化界面。

6.3.2 后端技术选型

(1) 语言与框架：推荐使用 Python 与 Django 或 Flask 框架来构建后端服务，这些框架对深度学习模型的集成支持较好。

(2) 数据存储：可以使用 MySQL 或 PostgreSQL 作为数据库，结合阿里云的 RDS 服务实现数据持久化存储；图像数据则可以使用阿里云 OSS 进行管理。

(3) API 服务：使用 Django Rest Framework (DRF) 或 GraphQL 实现 API 接口，方便前后端通信。

(4) 异步处理：使用 Celery 与 RabbitMQ 结合，实现模型的异步调用与任务管理，以应对高并发场景。

7. 结论与展望

本项目围绕肝脏肿瘤分割任务，结合深度学习技术，设计并实现了一个智能诊断系统。在研究过程中，我们完成了数据的预处理、模型的训练与优化、模型的评估与部署等多个环节。通过实验验证，基于 U-Net 架构的模型在肝脏肿瘤分割任务中表现出色，能够准确地从 CT 图像中分割出肿瘤区域。这一系统不仅展现了深度学习技术在医学影像分析中的强大潜力，也为实际临床诊断提供了一个有效的辅助工具。

智能诊断系统的实际应用价值主要体现在以下几个方面：首先，它能够显著提高医生的诊断效率，减少人为误差，从而为患者提供更快速、准确的诊疗建议。其次，系统能够对大量 CT 图像进行批量处理，尤其在肝脏肿瘤筛查等场景中具有广泛应用前景。最后，该系统的自动化和标准化处理流程也有助于提高医疗服务的一致性和可靠性。

展望未来，智能诊断系统仍有许多优化和扩展的空间。首先，模型可以进一步结合多模态数据（如 MRI、PET 等）以提升分割精度和诊断能力。其次，可以考虑引入更先进的深度学习架构^[5]，如 Transformer 或自监督学习方法，进一步提升模型的表现。此外，系统还可以扩展到更多的医学影像分析任务，如器官分割、病灶检测等，构建一个综合性的智能诊断平台。在应用层面，未来的研究还应关注如何更好地与临床工作流程融合，确保系统能够在实际应用中最大程度地发挥其价值。

参考文献

- [1] 康雨琪.人工智能对医疗健康行业的影响及应对[J].现代企业,2024,(08):77-79.
- [2] 杨磊,陈宽,陈丹丹.人工智能辅助 CT 诊断肝脏肿瘤与病理活检穿刺的一致性研究[J].中西医结合肝病杂志,2023,33(11):1022-1025.
- [3] Soler, L., et al. "3D image reconstruction for comparison of algorithm database: A patient specific anatomical and medical image database." IRCAD, Strasbourg, France, Tech. Rep (2010).
- [4] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. ArXiv, abs/1505.04597.
- [5] Khoshkhabar, M., Meshgini, S., Afrouzian, R., & Danishvar, S. (2023). Automatic Liver Tumor Segmentation from CT Images Using Graph Convolutional Network. *Sensors (Basel, Switzerland)*, 23.

附录

- 导库

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pydicom
import cv2
import tensorflow as tf
import shutil
```

- #1、数据预处理

- ##1.1 数据读取

```
data_path = r'C:\Users\chen\Desktop\人工智能实习报告
\3Dircadb1\3Dircadb1.1\PATIENT_DICOM'
#批量读取
image_slices = [pydicom.dcmread(os.path.join(data_path,file_name)) for file_name in
os.listdir(data_path)]
os.listdir(data_path)
image_slices.sort(key=lambda x:x.InstanceNumber)
#提取像素值
image_array = np.array([i.pixel_array for i in image_slices])
#可视化读取数据
j = 1
for i in range(0,6):
    plt.subplot(3,3,j)
    plt.imshow(image_array[i],cmap = 'gray')
    plt.axis('off')
```

```

    j += 1
plt.show()
## 1.2 CT 图数据增强: windowing 方法 (ct 对比增强)
#像素值
plt.hist(image_array.reshape(-1,1),bins = 50)
plt.show()
#windowing 自定义函数
def windowing(img>window_width>window_center):#需要增强的图片、窗宽、中心
    minWindow = float(window_center)-0.5*float(window_width)
    new_img = (img-minWindow)/float(window_width)
    new_img[new_img<0] = 0
    new_img[new_img>1] = 1
    return (new_img*255).astype('uint8')
img_ct = windowing(image_array,250,0)
#可视化读取数据
j = 1
for i in range(0,6):
    plt.subplot(3,3,j)
    plt.imshow(img_ct[i],cmap = 'gray')
    plt.axis('off')
    j += 1
plt.show()
## 1.3 直方图均衡化
img_ct.shape
clahe = cv2.createCLAHE(clipLimit = 2.0,tileGridSize=(8,8))
clahe.apply(img_ct[0])
# 自定义批量均衡化函数
def clahe_en(imgs):
    assert len(imgs.shape) == 3, "输入的图像应该是一个三维数组 (n, h, w), 其中 n 是
    图像数量。"
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    img_res = np.zeros_like(imgs)
    for i in range(len(imgs)):
        # 将图像转换为 uint8 类型, 并应用 CLAHE
        img_res[i, :, :] = clahe.apply(imgs[i, :, :].astype(np.uint8))
    # 返回归一化到 [0, 1] 之间的结果
    return img_res / 255.0
img_clahe = clahe_en(img_ct)

```

```

img_ct = windowing(image_array,250,0)
#可视化读取数据
j = 1
for i in range(0,6):
    plt.subplot(3,3,j)
    plt.imshow(img_clahe[i],cmap = 'gray')
    plt.axis('off')
    j += 1
plt.show()
##1.4 获取肿瘤部位 CT 图和掩膜图
data_path_mask = r'C:\Users\chen\Desktop\人工智能实习报告
\3Dircadb1\3Dircadb1.1\MASKS_DICOM\livertumor01'
#批量读取
tumor_slices = [pydicom.dcmread(os.path.join(data_path_mask,file_name)) for file_name in
os.listdir(data_path_mask)]
tumor_slices.sort(key=lambda x:x.InstanceNumber)
#提取像素值
tumor_array = np.array([i.pixel_array for i in tumor_slices])
#可视化读取数据
j = 1
for i in range(51,60):
    plt.subplot(3,3,j)
    plt.imshow(tumor_array[i],cmap = 'gray')
    plt.axis('off')
    j += 1
plt.show()
#一开始还没有肿瘤的掩膜图全部是黑色，对应的像素数组全为 0
#提取掩膜图的肿瘤部分
img_tumor = tumor_array[index]
#对增强后的 CT 图提取肿瘤部分
img_patient = img_clahe[index]
#可视化肿瘤数据
j = 1
for i in range(0,9):
    plt.subplot(3,3,j)
    plt.imshow(img_tumor[i],cmap = 'gray')
    plt.axis('off')
    j += 1

```

```

plt.show()
j = 1
for i in range(0,9):
    plt.subplot(3,3,j)
    plt.imshow(img_patient[i],cmap = 'gray')
    plt.axis('off')
    j += 1
plt.show()
##1.5 保留肿瘤数据
patient_save_path = r'C:\Users\chen\Desktop\人工智能实习报告\tmp\patient'
tumor_save_path = r'C:\Users\chen\Desktop\人工智能实习报告\tmp\tumor'
for path in [patient_save_path,tumor_save_path]:
    if os.path.exists(path):
        shutil.rmtree(path)
    os.makedirs(path)
#保存所有的肿瘤数据
liver_tumor_path = r'C:\Users\chen\Desktop\人工智能实习报告\3Dircadb1\3Dircadb1.1\MASKS_DICOM'
tumor_paths = [os.path.join(liver_tumor_path,i) for i in os.listdir(liver_tumor_path) if
'livertumor' in i]
#提取所有肿瘤数据
j = 0
for tumor_path in tumor_paths:
    print('正在处理第%d 个肿瘤'%j)
    #批量读取
    tumor_slices = [pydicom.dcmread(os.path.join(tumor_path,file_name)) for file_name in
os.listdir(tumor_path)]
    tumor_slices.sort(key=lambda x:x.InstanceNumber)
    #提取像素值
    tumor_array = np.array([i.pixel_array for i in tumor_slices])
    #提取掩膜图的肿瘤部分
    img_tumor = tumor_array[index]
    #对增强后的 CT 图提取肿瘤部分
    img_patient = img_clahe[index]
    for i in range(len(img_patient)):

plt.imshow(os.path.join(patient_save_path,f'{j}_{i}.jpg'),img_patient[i],cmap='gray')#CT 图

```

```

plt.imsave(os.path.join(tumor_save_path,f'{j}_{i}.jpg'),img_patient[i],cmap='gray')#掩膜图
    j+=1
##1.6 对所有病人的数据进行处理
for num in range(1, 21):
    print('正在处理第%d 位病人的数据' % num)
    # ---CT 图处理---
    data_path = fr'C:\Users\chen\Desktop\人工智能实习报告
\3Dircadb1\3Dircadb1.{num}\PATIENT_DICOM'
    # 批量读取
    image_slices = [pydicom.dcmread(os.path.join(data_path, file_name)) for file_name in
os.listdir(data_path)]
    image_slices.sort(key=lambda x: x.InstanceNumber)
    # 提取像素值
    image_array = np.array([i.pixel_array for i in image_slices])
    # CT 图增强
    img_ct = windowing(image_array, 250, 0)
    # 直方图均衡法
    img_clahe = clahe_en(img_ct)
    # ---掩模图处理---
    livertumor_path = fr'C:\Users\chen\Desktop\人工智能实习报告
\3Dircadb1\3Dircadb1.{num}\MASKS_DICOM'
    tumor_paths = [os.path.join(livertumor_path, i) for i in os.listdir(livertumor_path) if
'livertumor' in i]
    tumor_paths.sort()
    j = 0
    for tumor_path in tumor_paths:
        print('正在处理第%d 个肿瘤' % j)
        # 批量读取
        tumor_slices = [pydicom.dcmread(os.path.join(tumor_path, file_name)) for
file_name in os.listdir(tumor_path)]
        tumor_slices.sort(key=lambda x: x.InstanceNumber)
        # 提取像素值
        tumor_array = np.array([i.pixel_array for i in tumor_slices])
        # 确保 index 的维度与 tumor_array 和 img_clahe 匹配
        index = np.ones(tumor_array.shape[0], dtype=bool) # 示例: 创建一个全为
True 的布尔数组

```

```

        if len(index) == len(tumor_array) == len(img_clahe):
            img_tumor = tumor_array[index]
            img_patient = img_clahe[index]
            for i in range(len(img_patient)):
                plt.imsave(os.path.join(patient_save_path, f'{num}_{j}_{i}.jpg'),
img_patient[i], cmap='gray') # CT 图
                plt.imsave(os.path.join(tumor_save_path, f'{num}_{j}_{i}.jpg'),
img_tumor[i], cmap='gray') # 掩膜图
            else:
                print(f'Index length: {len(index)}, Tumor array length: {len(tumor_array)},
CT array length: {len(img_clahe)}')
                raise ValueError("Index length does not match tumor_array or img_clahe
length.")
        j += 1

```

- #2 #图像数据增强：定义 ImageDataGenerator 图片生成器、使用 flow_from_directory 方法图像增强

##2.1 定义图片生成器

```

import os
import shutil
import tensorflow as tf
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras import backend as K
from tensorflow.keras import *
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.preprocessing.image import ImageDataGenerator
generator_args = dict(
    rotation_range = 0.1,
    width_shift_range = 0.05,
    height_shift_range = 0.05,
    shear_range = 0.05,
    zoom_range = 0.05,
    horizontal_flip = False,
    vertical_flip = False,
)
ct_datagen = ImageDataGenerator(generator_args)
mask_datagen = ImageDataGenerator(generator_args)

```

##2.2 图像增强实现

#保存图像增强后的数据

save_datagen_path = r'C:\Users\chen\Desktop\人工智能实习报告\tmp2'

if os.path.exists(save_datagen_path):

shutil.rmtree(save_datagen_path)

os.makedirs(save_datagen_path)

#目标路径-ct 图的增强

generator_path = r'C:\Users\chen\Desktop\人工智能实习报告\tmp'

ct_generator = ct_datagen.flow_from_directory(

generator_path,

classes=['patient'],

class_mode=None,

color_mode = 'grayscale',

target_size = (512,512),

batch_size=2,

save_to_dir=save_datagen_path,

save_prefix='ct_',

seed = 123,

)

mask_generator = mask_datagen.flow_from_directory(

generator_path,

classes=['tumor'],

class_mode=None,

color_mode = 'grayscale',

target_size = (512,512),

batch_size=2,

save_to_dir=save_datagen_path,

save_prefix='mask_',

seed = 123,

)

train_generator = zip(ct_generator,mask_generator)

#可视化

for ct,mask in train_generator:

print(ct[0][:,:,0].shape,mask.shape)

plt.subplot(2,2,1)

plt.imshow(ct[0][:,:,0],cmap='gray')

plt.axis('off')

plt.subplot(2,2,2)

plt.imshow(mask[0][:,:,0],cmap='gray')

```

plt.axis('off')
plt.subplot(2,2,3)
plt.imshow(ct[1][:,:,0],cmap='gray')
plt.axis('off')
plt.subplot(2,2,4)
plt.imshow(mask[1][:,:,0],cmap='gray')
plt.axis('off')
break
plt.show()
##2.3 图像归一化
def adjust_data(ct,mask):
    ct /= 255.0
    mask /= 255.0
    mask[mask>0.5] = 1
    mask[mask<0.5] = 0
    return ct,mask
##2.4 整合图像增强的步骤
#自定义图像增强函数
def train_generator():
    #2.4.1. 定义图像生成器
    generator_args = dict(
        rotation_range = 0.1,
        width_shift_range = 0.05,
        height_shift_range = 0.05,
        shear_range = 0.05,
        zoom_range = 0.05,
        horizontal_flip = False,
        vertical_flip = False,
    )
    #2.4.2.分别对 ct 图和掩模图做图像增强
    ct_datagen = ImageDataGenerator(generator_args)
    mask_datagen = ImageDataGenerator(generator_args)
    #2.4.3.实践图像增强
    generator_path = r'C:\Users\chen\Desktop\人工智能实习报告\tmp'
    ct_generator = ct_datagen.flow_from_directory(
        generator_path,
        classes=['patient'],
        class_mode=None,
        color_mode = 'grayscale',

```

```

        target_size = (512,512),
        batch_size=2,
        save_to_dir=save_datagen_path,
        save_prefix='ct_',
        seed = 123,
    )
    mask_generator = mask_datagen.flow_from_directory(
        generator_path,
        classes=['tumor'],
        class_mode=None,
        color_mode = 'grayscale',
        target_size = (512,512),
        batch_size=2,
        save_to_dir=save_datagen_path,
        save_prefix='mask_',
        seed = 123,
    )
    train_generator = zip(ct_generator,mask_generator)#成组处理
#2.4.4.增强后的图片做归一化处理
    for (ct,mask) in train_generator:
        yield adjust_data(ct,mask)

```

● #3.构建模型

##3.1 搭建模型

```

import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPool2D, UpSampling2D,
concatenate
K.clear_session()
def u_net(input_size=(512, 512, 1)):
    # 输入层
    inputs = Input(input_size)
    # 卷积层
    conv1 = Conv2D(64, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(inputs)
    conv2 = Conv2D(64, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv1)
    # 池化层
    pool1 = MaxPool2D(pool_size=(2, 2))(conv2)
    # 卷积层

```

```

conv3 = Conv2D(128, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(pool1)
conv4 = Conv2D(128, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv3)
# 池化层
pool2 = MaxPool2D(pool_size=(2, 2))(conv4)
# 卷积层
conv5 = Conv2D(256, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(pool2)
conv6 = Conv2D(256, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv5)
# 池化层
pool3 = MaxPool2D(pool_size=(2, 2))(conv6)
# 卷积层
conv7 = Conv2D(512, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(pool3)
conv8 = Conv2D(512, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv7)
# 池化层
pool4 = MaxPool2D(pool_size=(2, 2))(conv8)
# 卷积层
conv9 = Conv2D(1024, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(pool4)
conv10 = Conv2D(1024, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv9)
# 反卷积
up1 = UpSampling2D((2, 2))(conv10)
# 卷积层
conv11 = Conv2D(512, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(concatenate([up1, conv8], axis=3))
conv12 = Conv2D(512, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv11)
# 反卷积
up2 = UpSampling2D((2, 2))(conv12)
# 卷积层
conv13 = Conv2D(256, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(concatenate([up2, conv6], axis=3))

```

```

conv14 = Conv2D(256, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv13)
# 反卷积
up3 = UpSampling2D((2, 2))(conv14)
# 卷积层
conv15 = Conv2D(128, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(concatenate([up3, conv4], axis=3))
conv16 = Conv2D(128, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv15)
# 反卷积
up4 = UpSampling2D((2, 2))(conv16)
# 卷积层
conv17 = Conv2D(64, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(concatenate([up4, conv2], axis=3))
conv18 = Conv2D(64, 3, activation='relu', padding='same',
kernel_initializer='he_normal')(conv17)
# 输出层
outputs = Conv2D(1, 1, activation='sigmoid')(conv18)
model = tf.keras.Model(inputs=inputs, outputs=outputs)
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
return model
model = u_net()
model.summary()

```

湖南工商大学《人工智能专业实习报告》课程设计评审表

姓 名	陈佳婷	学号	2227030079	专业班级	智媒体 2202 班
题 目：基于深度学习的肝脏肿瘤分割智能辅助诊断系统					
评 审 意 见	评审项目	成 绩			
	立题或选题（10 分）				
	获取信息能力（10 分）				
	方案的新颖性（10 分）				
	方案的综合性（20 分）				
	方案的内容（40 分）				
	文档的格式（10 分）				
	成绩总评 100%				
指导教师					
评审时间		2023 年 9 月 日			